

Variables.tf file in Terraform?

- variables.tf is a Terraform configuration file used to **declare input variables**.
- It's a best practice to keep all variable declarations in variables.tf file

terraform.tfvars file

The **terraform.tfvars** file is **used to assign values to the variables** declared in variables.tf.

It helps separate configuration from implementation, making the code reusable and modular.

variables.tf (file)	terraform.tfvars (file)
variables names are defined here	Variable values are defined here
Eg: Declares variable "region" {}.	Eg: it Assigns value for region = "uswest-2"

terraform.auto.tfvars files:

- It is a special file in Terraform used to automatically assign values to variables.
- Terraform automatically loads this file if it exists in the working directory.
- It works **similarly to terraform.tfvars**, but with a key difference, We don't need to explicitly specify this file during execution.

Sensitive Variables:

Sensitive variables can be marked as such to prevent their values from being displayed in the CLI output.

```
variable "db_password" { description
= "database password"
sensitive = true
}
```

What is the order of precedence for Terraform variables?

1. Command-line (-var, -var-file) **#First Priority**
2. *.auto.tfvars files
3. terraform.tfvars file
4. Environment variables (TF_VAR_)
5. Default values in variable block **#Last priority**

Terraform Variable Precedence with EC2 Example

Terraform variable precedence decides which value is used when the same variable is defined in multiple places. The order below shows the highest priority first, meaning it overrides all below it.

1 Command-line (-var,-var-file) – First Priority

If you pass variables directly in the Terraform command, they override everything else.

Example:

main.tf

```
provider "aws" {
  region = var.region
}

resource "aws_instance" "example" {
  ami          = var.ami
  instance_type = var.instance_type
}
```

variables.tf

```
variable "region" {
  default = "us-east-1"
}

variable "ami" {
  default = "ami-12345678"
}

variable "instance_type" {
  default = "t2.micro"
}
```

Command:

```
terraform apply -var="instance_type=t3.small" -var="region=us-west-2"
```

Here, t3.small and us-west-2 will be used, even if defaults or files say otherwise.

2. terraform*.auto.tfvars files

Any file ending in .auto.tfvars is automatically loaded without explicitly passing it.

Example:

```
prod.auto.tfvars
```

```
instance_type = "t3.medium"  
region       = "ap-south-1"
```

Command: terraform apply

t3.medium and ap-south-1 override defaults but will be overridden if -var is passed.

3. terraform.tfvars file

If present, Terraform automatically loads it (lower priority than *.auto.tfvars).

Example:

```
terraform.tfvars
```

```
instance_type = "t2.large"  
region       = "us-east-1"
```

Command: terraform apply

t2.large and eu-central-1 override defaults, but get overridden by .auto.tfvars or CLI vars.

4. Environment variables (TF_VAR_)

Any environment variable prefixed with TF_VAR_ is treated as a Terraform variable.

Example:

```
export TF_VAR_instance_type="m5.large"  
export TF_VAR_region="ca-central-1"
```

terraform apply

t3.small and us-east-1 unless overridden by files or CLI.

5. Default values in variable block – Last Priority

If no other source provides the value, Terraform uses the default defined in the variable block.

Example:

variables.tf

```
variable "instance_type" {  
  default = "t2.micro"  
}
```

```
variable "region" {  
  default = "us-east-1"  
}
```

Will only be used if nothing else is provided.

Output values

output block is used to **display specific values** after the execution of terraform apply.

This is useful for retrieving important information like instance public IPs, DNS names, or resource IDs.

```
# Configure AWS  
provider provider "aws"  
{  
  region = "us-east-  
1"  
}  
  
# Create an EC2 instance resource "aws_instance"  
"my_server_1" {  
  ami = "ami-04b4f1a9cf54c11d0" # Amazon  
Linux 2 (us-east-1) instance_type = "t2.micro"  
}  
  
# Output the public IP output "public_ip" {  
value =  
aws_instance.my_server_1.public_ip #reference to ec2 resource to get its public  
ip description = "The public IP address of the EC2 instance." }
```

After applying terraform will get the output of `aws_instance.my_server_1.public_ip` & prints the output on the screen like below.

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.  
Outputs:  
public_ip = "54.147.239.133"
```

Note: Output variables can also be stored in their own output variables file named **output.tf**